

Architecture of the Legacy XBOING Codebase

X11 Breakout Game, 1993–1996

Punt Labs — XBoing Modernization Project

May 2026 | Revision 1.1

Rev	Date	Description
1.0	2026-03-24	Initial legacy architecture document
1.1	2026-05-30	Fact-check against original / source: corrected line counts, window dimensions, function names, constants, file inventory

Contents

1	Introduction	3
2	Build System	3
2.1	Toolchain and Flags	3
2.2	Compile-Time Defines	3
2.3	Generated Files	3
2.4	Source File Inventory	4
3	Window System and Graphics	5
3.1	X11 Initialization Sequence	5
3.2	Window Hierarchy	5
3.3	Graphics Contexts	5
3.4	XPM Pixmap Loading and Rendering Pipeline	7
3.5	Colormap and Color Cycling	7
3.6	Font Handling	7
3.7	SFX Compositing	7
4	Audio System	7
4.1	Fork+Pipe Architecture	7
4.2	Platform Drivers	8
4.3	Sound Assets	9
5	Game State Machine	9
5.1	Mode Enumeration	9
5.2	State Transition Diagram	9
5.3	Event Loop	9
5.4	Frame Timing	10
6	Ball Physics and Collision	11
6.1	Ball Data Structure	11
6.2	Ball State Machine	11

6.3	Paddle Collision and Bounce Angle	11
6.4	Block Collision (4-Region System)	12
6.5	Ball-to-Ball Collision	12
6.6	Wall Collisions	13
7	Block System	13
7.1	Grid Structure	13
7.2	Block Type Catalog	13
8	Paddle and Gun	14
8.1	Paddle Sizes and Movement	14
8.2	Control Modes	14
8.3	Gun/Bullet System	15
9	Scoring and Bonuses	15
9.1	Score Computation	15
9.2	Bonus Sequence	15
10	Level System	15
10.1	Level File Format	15
10.2	Background Rotation	16
10.3	Bonus Block Spawning	16
11	Save/Load and High Scores	16
11.1	Save File Format	16
11.2	High Score File Format	17
12	Level Editor	17
13	Module Dependency Map	17
14	Code Metrics	17
14.1	Coupling Characteristics	17
15	Known Issues and Technical Debt	19
A	Command-Line Options	20
B	Initialization Sequence	20
C	Resource Lifecycle Pairs	21

1 Introduction

XBOING is an X11 breakout-style game written by Justin C. Kibell between 1993 and 1996. It was distributed through the X Consortium under a permissive license. The game targets Unix workstations running the X Window System with PseudoColor visuals—the dominant display configuration of early-1990s SPARC, SGI, and HP-UX machines.

The codebase consists of 29 C source files (18,606 lines excluding tests), 33 headers (2,953 lines), and 12 platform-specific audio drivers (2,882 lines). It links against `libXpm`, `libX11`, and `libm`. There is no external game engine, framework, or toolkit; the author built directly on top of raw Xlib.

This document records the legacy architecture in sufficient detail for two audiences:

1. **Contributors** who need to understand the original design before modifying the modernized version.
2. **Researchers** studying AI-assisted modernization of legacy C code, who need a “before” baseline to compare against the SDL2-based “after.”

All section references marked (Spec §N) point to the corresponding section in `docs/SPECIFICATION.md`.

2 Build System

2.1 Toolchain and Flags

The original build used `Imakefile` (X Consortium’s build generator) to produce a platform-specific `Makefile`. The modernized legacy build uses a hand-written `Makefile` with the settings in table 1.

Table 1: Compiler and linker configuration.

Setting	Value
Compiler	<code>gcc</code>
CFLAGS	<code>-O2 -Wall -Wno-unused-result -Wno-format-overflow -Wno-format-truncation</code>
Include paths	<code>-I./include -I/usr/include/X11</code>
Libraries	<code>-lXpm -lX11 -lm</code>

Three warning suppressions are required because the original code uses `sprintf()` with unchecked return values and fixed-size buffers throughout.

2.2 Compile-Time Defines

All file system paths are baked in at compile time via `-D` flags (table 2). Each has a corresponding runtime environment variable override.

The original source used a `NeedFunctionPrototypes` guard for dual K&R / ANSI prototype support—every public function declaration was wrapped in `#if NeedFunctionPrototypes` conditionals, a pattern common in X Consortium code of this era. These guards have been removed from the archived source.

2.3 Generated Files

Two files are generated during build:

Table 2: Compile-time defines and their runtime overrides (Spec §1).

Define	Default	Env Override
HIGH_SCORE_FILE	./xboing.scr	XBOING_SCORE_FILE
LEVEL_INSTALL_DIR	../levels	XBOING_LEVELS_DIR
SOUNDS_DIR	../sounds	XBOING_SOUND_DIR
README_FILE	./docs/problems.doc	—
AUDIO_AVAILABLE	True	—
AUDIO_FILE	audio/LINUXaudio.c	—

version.c

Produced by `version.sh`. Contains four symbols: `dateString` (build timestamp), `whoString` (build user), `machineString` (architecture from `uname -a`), and `buildNum` (auto-incrementing counter stored in `.version`).

audio.c

A symlink to the active audio driver, defaulting to `audio/LINUXaudio.c`. Swapping the symlink target was the mechanism for cross-platform audio.

2.4 Source File Inventory

The build compiles 29 `.c` files (listed in table 3) and links them into a single `xboing` binary.

Table 3: Source files with line counts (excluding tests and stubs).

File	Lines	Role
<code>blocks.c</code>	2,631	Block grid, 30 types, collision regions
<code>ball.c</code>	2,099	Ball physics, collision detection
<code>main.c</code>	1,768	Event loop, state machine, input
<code>editor.c</code>	1,198	Level editor
<code>init.c</code>	1,071	X11 setup, colormap, fonts
<code>highscore.c</code>	1,065	Score file I/O, display
<code>bonus.c</code>	775	Bonus sequence
<code>presents.c</code>	701	Credits/splash sequence
<code>file.c</code>	703	Level file parsing, save/load
<code>gun.c</code>	642	Bullet system
<code>stage.c</code>	581	Window hierarchy creation
<code>level.c</code>	511	Level management, timer
<code>misc.c</code>	499	Drawing utilities
<code>intro.c</code>	469	Introduction screen
<code>eyedude.c</code>	402	Animated enemy character
<code>dialogue.c</code>	393	Modal input dialogs
<code>sfx.c</code>	383	Visual special effects
<code>keys.c</code>	384	Key bindings display
<code>paddle.c</code>	357	Paddle control
<code>demo.c</code>	340	Demo mode
<code>keysedit.c</code>	327	Editor key bindings display
<code>inst.c</code>	287	Instructions screen
<code>special.c</code>	253	Power-up state management
<code>preview.c</code>	241	Level preview screen

File	Lines	Role
<code>score.c</code>	219	Score digit display
<code>mess.c</code>	159	Message bar
<code>error.c</code>	148	Error/warning output
<code>version.c</code>	—	Generated (build metadata)
<code>audio.c</code>	—	Symlink to platform driver
Total	18,606	(27 files, excl. generated)

3 Window System and Graphics

3.1 X11 Initialization Sequence

`InitialiseGame()` in `init.c` performs 33 ordered initialization steps (Spec §16). The sequence has implicit dependencies: colormap must precede color allocation, which must precede window creation, which must precede GC creation. Reordering any step risks segfaults or blank windows.

The visual selection algorithm tries, in order: `PseudoColor`, `DirectColor`, `TrueColor`. `PseudoColor` was the norm on 1993 workstations; modern systems universally provide `TrueColor`.

3.2 Window Hierarchy

The application creates 11 windows: one top-level `mainWindow` and 10 child windows for distinct UI regions (table 4). This was standard Xlib practice—each sub-window provides its own clipping rectangle, event routing, and background.

Table 4: Window hierarchy with positions and dimensions (from `stage.c`; `SPECIFICATION.md` lists older 565×710 dimensions).

Window	Size (px)	Position	Purpose
<code>mainWindow</code>	575 × 720	(0,0)	Top-level container
<code>playWindow</code>	495 × 580	(35,60)	Gameplay area
<code>bufferWindow</code>	495 × 580	(35,60)	Offscreen double buffer
<code>scoreWindow</code>	224 × 42	(35,10)	Score display
<code>levelWindow</code>	286 × 52	(284,5)	Level, lives, ammo
<code>messWindow</code>	247 × 30	(35,655)	Status messages
<code>specialWindow</code>	180 × 35	(292,655)	Active specials
<code>timeWindow</code>	61 × 35	(477,655)	Timer display
<code>blockWindow</code>	120 × 580	(545,60)	Editor palette
<code>typeWindow</code>	120 × 35	(545,650)	Editor block type
<code>inputWindow</code>	380 × 120	(92,295)	Modal input dialog

The `bufferWindow` is never mapped (made visible). It serves as an offscreen pixmap for double buffering—SFX transitions copy from `bufferWindow` to `playWindow` in strips or chunks.

3.3 Graphics Contexts

Six GCs control all drawing operations (table 5). The X11 GC is a server-side object that stores drawing parameters (function, foreground/background, fill style, clip mask). Switching drawing modes requires selecting a different GC, not modifying one in place.

The AND/OR pair implements sprite transparency: AND the mask (clearing the sprite area),

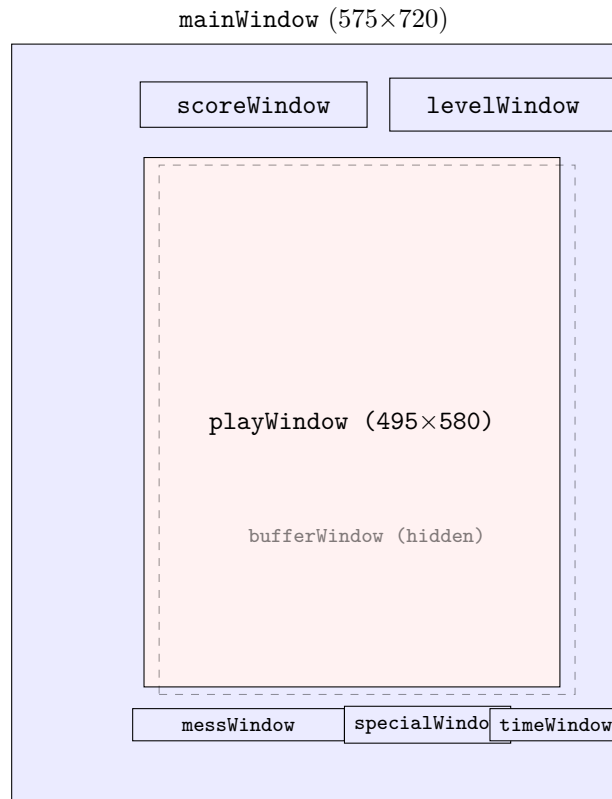


Figure 1: X11 window hierarchy. The `bufferWindow` (dashed) is an unmapped offscreen surface used for SFX compositing. Editor windows (`blockWindow`, `typeWindow`) omitted—they extend the main window width when the editor is active.

Table 5: Graphics contexts and their X11 raster operations (Spec §2).

GC	X Function	Fill Style	Purpose
<code>gccopy</code>	<code>GXcopy</code>	Solid	Standard blitting
<code>gc</code>	<code>GXcopy</code>	FillTiled	Tiled pattern fills
<code>gcxor</code>	<code>GXxor</code>	Solid	Reversible drawing (erase by redraw)
<code>gcand</code>	<code>GXand</code>	Solid	Mask application (<code>fg=0</code> , <code>bg=~0</code>)
<code>gcor</code>	<code>GXor</code>	Solid	Image compositing (OR combine)
<code>gcsfx</code>	<code>GXcopy</code>	Solid	Special effects

then OR the sprite image (compositing it onto the background). This three-step blit pattern predates alpha blending.

3.4 XPM Pixmap Loading and Rendering Pipeline

All graphics are XPM files—a C-header format encoding indexed-color images. The `XpmCreatePixmapFromData()` function (from `libXpm`) returns both a `Pixmap` (image data) and a shape `Pixmap` (1-bit transparency mask). The rendering pipeline is:

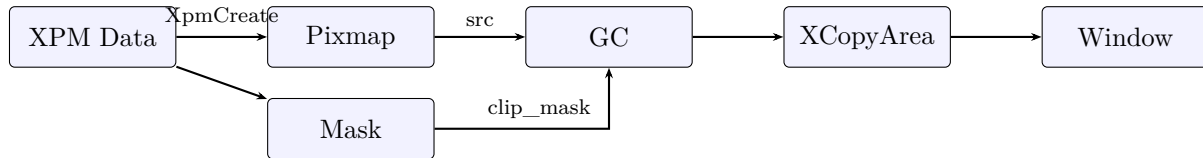


Figure 2: Rendering pipeline: XPM source → Pixmap + Mask → GC with clip mask → XCopyArea → destination Window.

3.5 Colormap and Color Cycling

The game allocates a private `PseudoColor` colormap with 8 named colors (red, tan, yellow, green, purple, blue, white, black) and two cycling arrays:

- `reds[7]`: shades from `#f00` to `#300` (indices 0–6)
- `greens[7]`: shades from `#0f0` to `#030` (indices 0–6)

These arrays drive the border glow animation, which cycles every 40 frames. When the `noWalls` power-up is active, the border switches to the green array.

3.6 Font Handling

Four Adobe Helvetica bitmap fonts are loaded via XLFD (X Logical Font Description) patterns (table 6). If any pattern fails to match, the code falls back to the `"fixed"` font.

Table 6: Font specifications (Spec §2).

Variable	Size	Weight	Usage
<code>titleFont</code>	24pt	Bold	Title text
<code>textFont</code>	18pt	Medium	General text
<code>dataFont</code>	14pt	Bold	Data and intro text
<code>copyFont</code>	12pt	Medium	Copyright, specials

3.7 SFX Compositing

Five visual effects use the `bufferWindow` as a source and `playWindow` as a destination (table 7). All require backing store (`DoesBackingStore() == Always`); without it, effects are silently disabled.

4 Audio System

4.1 Fork+Pipe Architecture

The audio system uses a Unix process model: `SetUpAudioSystem()` calls `fork()` to create a child process. Parent and child communicate over a unidirectional pipe. The parent writes sound

Table 7: Visual special effects modes (Spec §10).

ID	Constant	Name	Technique
1	SFX_SHAKE	Shake	Move <code>playWindow</code> ± 3 px randomly every 5 frames
2	SFX_FADE	Fade	Draw black grid lines at 12 px spacing
3	SFX_BLIND	Blind	Copy buffer in 8 vertical strips
4	SFX_SHATTER	Shatter	10 $\times$ 10 scatter grid, randomized chunk copy
5	SFX_STATIC	Static	Placeholder (no animation)

file names (null-terminated strings) to the pipe; the child reads them, opens the corresponding `.au` file, and writes raw PCM data to `/dev/dsp` via the OSS (Open Sound System) interface.

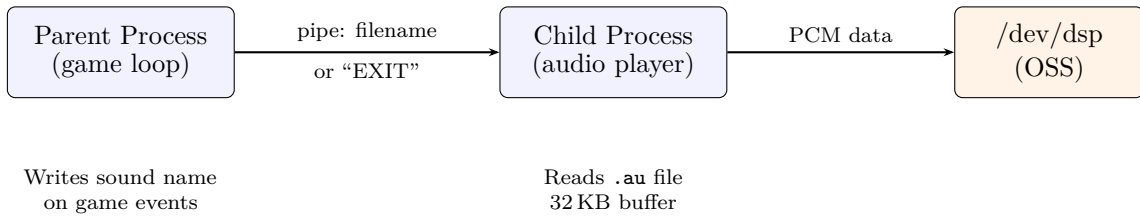


Figure 3: Audio fork+pipe architecture. The child process blocks on pipe reads when idle. Termination is signaled by writing the string “EXIT.”

The 32 KB buffer (`SBUF_SIZE = 32`, `BUFFER_SIZE = 1024  $\times$  32`) is used for reading `.au` file data. After playing each file, the child issues `SNDCTL_DSP_SYNC` to flush the device. Volume control is a no-op in the Linux driver.

4.2 Platform Drivers

Twelve drivers exist in the `audio/` directory (table 8). The active driver is selected at build time by creating the `audio.c` symlink. All drivers implement the same 6-function API: `SetUpAudioSystem()`, `FreeAudioSystem()`, `playSoundFile()`, `audioDeviceEvents()`, `SetMaximumVolume()`, `GetMaximumVolume()`.

Table 8: Audio driver inventory (2,882 lines total) (Spec §3).

Driver File	Platform	Lines
<code>LINUXaudio.c</code>	Linux OSS (<code>/dev/dsp</code>)	234
<code>LINUXaudio2.c</code>	Linux alternate	292
<code>ALSAaudio.c</code>	ALSA	76
<code>SUNaudio.c</code>	SunOS/Solaris	389
<code>SGIaudio.c</code>	SGI IRIX	305
<code>HPaudio.c</code>	HP-UX	200
<code>BSDaudio.c</code>	BSD	323
<code>SVR4audio.c</code>	System V Release 4	222
<code>AFaudio.c</code>	AudioFile (AF) protocol	322
<code>RPLAYaudio.c</code>	RPLAY (remote audio)	187
<code>NCDaudio.c</code>	Network Computing Devices	229
<code>NOaudio.c</code>	Null/silent	103

4.3 Sound Assets

The game ships 46 .au (Sun Audio) files in `sounds/`. Sound names are passed as bare strings (e.g., "boing", "applause") and the driver constructs the full path as `{SOUNDS_DIR}/{name}.au`.

5 Game State Machine

5.1 Mode Enumeration

The game has 16 modes, defined as integer constants in `main.h` (table 9). The active mode is stored in the global variable `mode`; the previous mode in `oldMode`.

Table 9: Game mode constants (Spec §4).

ID	Constant	Description
0	MODE_NONE	Uninitialized
1	MODE_HIGHSCORE	High score display
2	MODE_INTRO	Introduction/splash
3	MODE_GAME	Active gameplay
4	MODE_PAUSE	Paused
5	MODE_BALL_WAIT	Waiting for ball activation
6	MODE_WAIT	Generic wait
7	MODE_BONUS	Level-complete bonus sequence
8	MODE_INSTRUCT	Instructions display
9	MODE_KEYS	Key bindings display
10	MODE_PRESENTS	Credits/splash (startup)
11	MODE_DEMO	Auto-play demo
12	MODE_PREVIEW	Level preview
13	MODE_DIALOGUE	Modal dialog
14	MODE_EDIT	Level editor
15	MODE_KEYSEEDIT	Editor key bindings

5.2 State Transition Diagram

5.3 Event Loop

The event loop in `handleEventLoop()` (`main.c`) runs a non-blocking poll during gameplay and a blocking wait during pause or iconification:

Listing 1: Event loop structure (simplified).

```
void handleEventLoop(Display *display) {
    /* Wait for MapNotify, grab pointer */
    while (True) {
        audioDeviceEvents();
        if (!iconified && mode != MODE_PAUSE) {
            XPending(display); /* non-blocking */
            if (mode != MODE_DIALOGUE) frame++;
        } else {
            XPeekEvent(display, &event); /* blocks */
            XPending(display);
        }
        /* dispatch: KeyPress, ButtonPress, Expose, Map/Unmap */
        if (!iconified) {
            if (mode == MODE_GAME || mode == MODE_BALL_WAIT)
```


6 Ball Physics and Collision

6.1 Ball Data Structure

Each ball is represented by a `BALL` struct (table 11). The game supports up to `MAX_BALLS = 5` simultaneous balls.

Table 11: Ball structure fields (Spec §5).

Field	Type	Description
<code>ballx, bally</code>	<code>int</code>	Current center position (pixels)
<code>oldx, oldy</code>	<code>int</code>	Previous position
<code>dx, dy</code>	<code>int</code>	Velocity components (px/frame)
<code>radius</code>	<code>float</code>	Geometric radius
<code>mass</code>	<code>float</code>	Mass (1.0–3.0)
<code>active</code>	<code>int</code>	True if ball in use
<code>ballState</code>	<code>enum</code>	Current state (8 values, see section 6.2)
<code>slide</code>	<code>int</code>	Animation frame index (0–4)
<code>lastPaddleHitFrame</code>	<code>int</code>	Frame of last paddle contact
<code>waitMode, waitingFrame, newMode</code>	various	Wait/transition control

Key constants: `BALL_WIDTH = 20 px`, `BALL_HEIGHT = 19 px`, `MAX_X_VEL = MAX_Y_VEL = 14 px/frame`, `MIN_DX_BALL = MIN_DY_BALL = 2 px/frame`.

6.2 Ball State Machine

Table 12: Ball states and lifecycle (Spec §5).

Value	State	Description
0	<code>BALL_POP</code>	Dying animation (8 frames)
1	<code>BALL_ACTIVE</code>	In play, moving
2	<code>BALL_STOP</code>	Dead
3	<code>BALL_CREATE</code>	Birth animation (8 frames)
4	<code>BALL_DIE</code>	About to die
5	<code>BALL_WAIT</code>	Transitioning between modes
6	<code>BALL_READY</code>	On paddle, awaiting launch
7	<code>BALL_NONE</code>	Null/unused

Lifecycle: `BALL_NONE` → `BALL_CREATE` (8-frame birth) → `BALL_READY` (auto-launch after 3,000 ms) → `BALL_ACTIVE` → `BALL_DIE/BALL_POP` (8-frame death) → `BALL_NONE`.

6.3 Paddle Collision and Bounce Angle

The ball-paddle collision test is a geometric line intersection. The paddle sits at $y = \text{PLAY_HEIGHT} - \text{DIST_BASE} - 2$ (where `DIST_BASE = 30`). Two cases:

Case A (vertical ball, $dx = 0$): check if `ballx` falls within paddle bounds.

Case B (diagonal ball, $dx \neq 0$): compute the trajectory line $y = \alpha x + \beta$, find the intersection $x_H = (\text{paddleLine} - \beta)/\alpha$, and check if x_H is within paddle bounds.

The bounce angle uses trigonometric reflection:

$$V_s = \sqrt{V_x^2 + V_y^2} \quad (1)$$

$$\alpha = \arctan\left(\frac{V_x}{-V_y}\right) \quad (2)$$

$$\beta = \arctan\left(\frac{\text{hitPos}}{\text{padSize}}\right) \quad (3)$$

$$\gamma = 2\beta - \alpha \quad (4)$$

$$V'_x = V_s \sin \gamma + \frac{\text{paddleDx}}{10} \quad (5)$$

$$V'_y = -V_s \cos \gamma \quad (6)$$

The term $\text{paddleDx}/10$ in eq. (5) adds paddle momentum to the ball, allowing the player to influence ball direction through paddle movement. Minimum upward velocity is enforced: $dy' \geq -\text{MIN_DY_BALL}$.

6.4 Block Collision (4-Region System)

Each block is divided into 4 triangular collision regions using `XPolygonRegion()`. The ball's bounding box is tested against each region via `XRectInRegion()` to determine bounce direction.

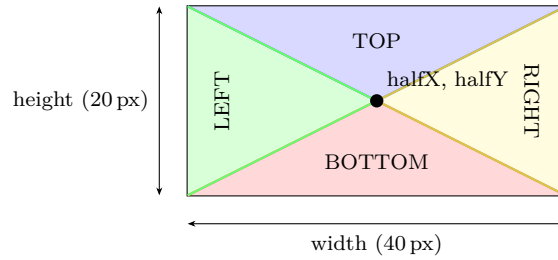


Figure 5: Block collision regions. Each block is divided into 4 triangles meeting at the center. The region hit determines bounce direction: TOP/BOTTOM reverses dy ; LEFT/RIGHT reverses dx . Corner hits combine flags.

Region constants: `REGION_TOP` = 1, `REGION_BOTTOM` = 2, `REGION_LEFT` = 4, `REGION_RIGHT` = 8. Corner hits combine via bitwise OR.

Detection priority depends on ball direction: if $|dx| > |dy|$, horizontal regions are checked first; otherwise, vertical regions take priority.

6.5 Ball-to-Ball Collision

Ball-to-ball collision uses a quadratic prediction to find the collision time within the current frame. Given position delta $\mathbf{p}$ and velocity delta $\mathbf{v}$ between two balls:

$$v^2 = v_x^2 + v_y^2 \quad (7)$$

$$r^2 = (r_1 + r_2)^2 \quad (8)$$

$$D = v^2 \cdot r^2 - (v_x p_y - v_y p_x)^2 \quad (9)$$

If $D \geq 0$ and $v^2 > \epsilon$ (where $\epsilon = \sqrt{1.4 \times 10^{-45}}$), a collision is predicted. The collision time $t_{\min}$ is computed via the quadratic formula; if $0 \leq t_{\min} \leq 1$, collision occurs this frame.

The elastic response uses mass-based velocity exchange:

$$m_r = m_1/m_2 \quad (10)$$

$$k = \frac{-2(\mathbf{v} \cdot \mathbf{p})}{1 + m_r} \quad (11)$$

$$\Delta \mathbf{v}_1 = k \cdot \hat{\mathbf{p}}, \quad \Delta \mathbf{v}_2 = -k \cdot m_r \cdot \hat{\mathbf{p}} \quad (12)$$

6.6 Wall Collisions

Table 13: Boundary collision behavior (Spec §5).

Boundary	Normal Mode	noWalls Mode
Left ($x < \text{BALL_WC}$)	Reverse dx : $dx = dx $	Wrap right: $x = \text{PLAY_WIDTH} - \text{BALL_WC}$
Right ($x > \text{PW} - \text{BALL_WC}$)	Reverse dx : $dx = - dx $	Wrap left: $x = \text{BALL_WC}$
Top ($y < \text{BALL_HC}$)	Reverse dy : $dy = dy $	Same (reverse dy)
Bottom ($y > \text{PH} + 2 \cdot \text{BH}$)	Ball dies	Ball dies

7 Block System

7.1 Grid Structure

The block grid is an 18×9 array ($\text{MAX_ROW} \times \text{MAX_COL}$) of `struct aBlock`. Each cell is 40×20 px ($\text{BLOCK_WIDTH} \times \text{BLOCK_HEIGHT}$), except `BLACK_BLK` (50×30) and `BOMB_BLK/DEATH_BLK` (30×30). The bottom 3 rows are reserved for the paddle area, leaving 15 playable rows.

The `aBlock` struct has 29 fields including pixel position, collision regions (4 `X11 Region` objects), animation state, explosion state, ball hit tracking, and type-specific counters.

7.2 Block Type Catalog

Table 14: Complete block type table (30 types) (Spec §6).

ID	Enum	Char	Size	Points	Behavior
0	RED_BLK	r	40×20	100	Standard
1	BLUE_BLK	b	40×20	110	Standard
2	GREEN_BLK	g	40×20	120	Standard
3	TAN_BLK	t	40×20	130	Standard
4	YELLOW_BLK	y	40×20	140	Standard
5	PURPLE_BLK	p	40×20	150	Standard
6	BULLET_BLK	B	40×20	50	Ammo pickup
7	BLACK_BLK	w	50×30	0	Indestructible wall
8	COUNTER_BLK	0-5	40×20	200	Multi-hit (0-5)
9	BOMB_BLK	X	30×30	50	Chain explosion
10	DEATH_BLK	D	30×30	0	Instant ball kill
11	REVERSE_BLK	R	33×16	100	Reverse controls
12	HYPERSPACE_BLK	H	31×31	100	Teleport ball
13	EXTRABALL_BLK	L	30×19	100	Extra life
14	MGUN_BLK	M	35×15	100	Machine gun

ID	Enum	Char	Size	Points	Behavior
15	WALLOFF_BLK	W	27×23	100	Disable walls
16	MULTIBALL_BLK	m	40×20	100	Split ball
17	STICKY_BLK	s	32×27	100	Sticky paddle
18	PAD_SHRINK_BLK	<	40×15	100	Shrink paddle
19	PAD_EXPAND_BLK	>	40×15	100	Expand paddle
20	DROP_BLK	d	40×20	$(18 - r) \times 100$	Falls when hit
21	MAXAMMO_BLK	c	40×20	50	Max ammo
22	ROAMER_BLK	+	25×27	400	Mobile roaming
23	TIMER_BLK	T	21×21	100	+20 seconds
24	RANDOM_BLK	?	40×20	0	Random on load
25	DYNAMITE_BLK	—	29×15	0	Chain explosion (bonus)
26	BONUSX2_BLK	—	27×27	0	2× multiplier (bonus)
27	BONUSX4_BLK	—	27×27	0	4× multiplier (bonus)
28	BONUS_BLK	—	27×27	0	Bonus coin (bonus)
29	BLACKHIT_BLK	—	50×30	0	Wall hit indicator
−2	NONE_BLK	.	—	—	Empty cell
−1	KILL_BLK	—	—	—	Destroyed marker

Types 0–24 are “static” types placeable in level files; types 25–29 are runtime-only (spawned during gameplay as bonus drops).

8 Paddle and Gun

8.1 Paddle Sizes and Movement

Three paddle sizes (table 15) with a fixed velocity of `PADDLE_VEL` = 10 px/frame. The paddle bottom edge sits `DIST_BASE` = 30 px above the play area floor.

Table 15: Paddle size constants (Spec §7).

Constant	ID	Width (px)	Half-Width
PADDLE_SMALL	4	40	20
PADDLE_MEDIUM	5	50	25
PADDLE_HUGE	6	70	35

Transitions: `PAD_SHRINK_BLK` moves `HUGE` → `MEDIUM` → `SMALL`. `PAD_EXPAND_BLK` moves `SMALL` → `MEDIUM` → `HUGE`. Ball death resets to `HUGE` (two expand calls). New game starts at `HUGE`.

8.2 Control Modes

- **Keyboard** (`CONTROL_KEYS` = 0): Left/Right arrows or J/L keys. Sets `paddleMotion` to −1 (left), 0 (stop), or +1 (right).
- **Mouse** (`CONTROL_MOUSE` = 1): Paddle follows pointer via `ObtainMousePosition()`. Computes `paddleDx` delta.

Toggle via G key. Special states: `reverseOn` inverts controls (set by `REVERSE_BLK`, reset on ball death); `stickyOn` makes ball stick to paddle (set by `STICKY_BLK`, auto-launches after 3,000 ms).

8.3 Gun/Bullet System

The bullet system supports up to `MAX_BULLETS = 20` ammo and `MAX_MOVING_BULLETS = 40` in-flight projectiles. Bullets travel upward at $dy = -7$ px/frame (`BULLET_DY`). Each new level resets ammo to `NUMBER_OF_BULLETS_NEW_LEVEL = 4`.

Normal gun fires 1 bullet at paddle center. Machine gun (activated by `MGUN_BLK`) fires 2 bullets offset by $\pm \text{paddleSize}/3$. Special blocks require `SHOTS_TO_KILL_SPECIAL = 3` bullet hits. `HYPERSPACE_BLK` and `BLACK_BLK` are immune to bullets.

9 Scoring and Bonuses

9.1 Score Computation

`ComputeScore()` applies active multipliers: if `x2Bonus` is active (set by collecting `BONUSX2_BLK`), $\text{inc} \times 2$; else if `x4Bonus` is active (set by collecting `BONUSX4_BLK`), $\text{inc} \times 4$. Collecting one multiplier deactivates the other. Additional score events: `PADDLE_HIT_SCORE = 10` per paddle contact, `EYEDUDE_HIT_BONUS = 10,000` per enemy kill.

Extra lives are awarded every `NEW_LIVE_SCORE_INC = 100,000` points, capped at `MAX_LIVES = 6`. Game starts with `START_LIVES = 3`.

9.2 Bonus Sequence

When a level is completed, a 10-state bonus tally sequence runs (table 16). The sequence awards:

1. **Bonus coins:** if `count > 8` (`MAX_BONUS`), award 50,000 (`SUPER_BONUS_SCORE`); else `count` $\times$ 3,000 (`BONUS_COIN_SCORE`).
2. **Level bonus:** $100 \times \text{levelNumber}$ (`LEVEL_SCORE`).
3. **Bullet bonus:** `remaining` $\times$ 500 (`BULLET_SCORE`).
4. **Time bonus:** `secondsLeft` $\times$ 100 (`TIME_BONUS`).

Table 16: Bonus sequence states (Spec §9).

State	Action
<code>BONUS_TEXT</code>	Display bonus screen title
<code>BONUS_SCORE</code>	Show congratulations
<code>BONUS_BONUS</code>	Tally collected bonus coins
<code>BONUS_LEVEL</code>	Level completion bonus
<code>BONUS_BULLET</code>	Remaining bullet bonus
<code>BONUS_TIME</code>	Time bonus calculation
<code>BONUS_HSCORE</code>	High score ranking check
<code>BONUS_END_TEXT</code>	“Prepare for next level”
<code>BONUS_WAIT</code>	Wait for frame transition
<code>BONUS_FINISH</code>	Complete, start next level

10 Level System

10.1 Level File Format

Level files are plain text stored in `levels/levelNN.data` (`NN = 01–80`, zero-padded). The format is 17 lines:

Listing 2: Level file format (level01.data example).

```
Genesis
120
.....
.....
rrrrrrrrr
bbbbbbbbb
ggggggggg
ttttttttt
.....
.....
000000000
yyyyyyyyy
ppppppppp
B...B...B
.....
.....
.....
```

Line 1: level title (string). Line 2: time limit (integer seconds). Lines 3–17: 15 rows of 9 characters, each character mapping to a block type (see table 14, “Char” column). The character . denotes an empty cell.

The game ships 83 level files: `level01.data` through `level80.data`, plus `demo.data`, `editor.data`, and `new.data`. Levels wrap via level mod 80.

10.2 Background Rotation

Each level cycles through backgrounds `BACKGROUND_2` through `BACKGROUND_5`, wrapping at 6 back to 2. Six background XPM pixmaps plus a “space” pattern are available.

10.3 Bonus Block Spawning

During `MODE_GAME`, random bonus blocks spawn at intervals of `frame + (rand() mod 2000)`. A 27-case switch selects the block type, with `BONUS_BLK` (cases 0–7) being the most common. Conditional guards prevent duplicate multiplier blocks.

11 Save/Load and High Scores

11.1 Save File Format

The save system writes two files:

`~/.xboing-saveinfo`

Binary serialization of `saveGameStruct` (9 fields, 36 bytes). Fields: `version` (`SAVE_VERSION = 2`), `score`, `level`, `levelTime`, `gameTime`, `livesLeft`, `startLevel`, `paddleSize`, `numBullets`.

`~/.xboing-savelevel`

Text file in the same format as level files.

Serialization uses raw `fread()/fwrite()`—no byte-order conversion, no padding protection. Auto-save triggers every `SAVE_LEVEL = 5` levels.

11.2 High Score File Format

Two score files exist: a global file (`.xboing.scr`, with file locking) and a personal file (`~/xboing-scores`, without locking).

The binary format consists of:

Table 17: High score file layout (684 bytes on 32-bit) (Spec §13).

Section	Size (32-bit)	Contents
<code>highScoreHeader</code>	84 bytes	<code>version</code> (4B, <code>htonl</code>), <code>masterText</code> [80]
<code>highScoreEntry</code> $\times$ 10	600 bytes	<code>score</code> , <code>level</code> (4B each, <code>htonl</code>), <code>gameTime</code> , <code>time</code> (<code>time_t</code> , 4B on 32-bit)

All numeric fields use network byte order (`htonl()`/`ntohl()`). File locking uses `lockf()` (`F_LOCK`/`F_ULOCK`) or `flock()` (`LOCK_EX`/`LOCK_UN`), configurable via `USE_FLOCK` or `NO_LOCKING` defines. Scores are sorted by descending value via bubble sort. The rank-1 holder sets a “Boing Master” wisdom text.

12 Level Editor

The built-in level editor (1,198 lines in `editor.c`) adds a tool palette window (`blockWindow`, 120px wide) to the right of the main window. It operates in 5 states: `EDIT_LEVEL` (load), `EDIT_NONE` (idle editing), `EDIT_TEST` (play-test), `EDIT_WAIT` (transition), and `EDIT_FINISH` (exit).

Grid constraints: 15 editable rows (`MAX_ROW_EDIT = MAX_ROW - 3`), 9 columns. Mouse controls: `Button1` draws, `Button2` erases, `Button3` queries `hitPoints`.

Board manipulation operations: `FlipBoardHorizontal()` (mirror L/R), `FlipBoardVertical()` (mirror T/B), `ScrollBoardHorizontal()` (rotate left with wrap), `ScrollBoardVertical()` (rotate up with wrap).

Play-test mode saves the editor level to a temp file in `/tmp/`, initializes with 3 lives, 0 score, HUGE paddle, and 4 bullets, then runs the game loop on that level. On exit, the original level is restored and the temp file is deleted.

13 Module Dependency Map

14 Code Metrics

14.1 Coupling Characteristics

The codebase exhibits several coupling patterns typical of 1990s C game development:

Global state via `extern`

62 `extern` declarations across 33 headers. Key shared globals include: `mode`, `frame`, `gameActive` (`main.h`); `display`, `mainWindow`, `playWindow`, and all 6 GCs (`init.h`/`stage.h`); `blocks`[] [] array (`blocks.h`).

X11 type proliferation

The `Display*` pointer is passed to virtually every function. X11 types (`Window`, `Pixmap`, `GC`, `Region`, `Colormap`) are used directly in game logic rather than abstracted behind an interface.

Module-as-singleton

Each `.c` file uses `static` variables for persistent state. This is a form of encapsulation, but it means module state cannot be inspected or reset without calling module-specific functions.

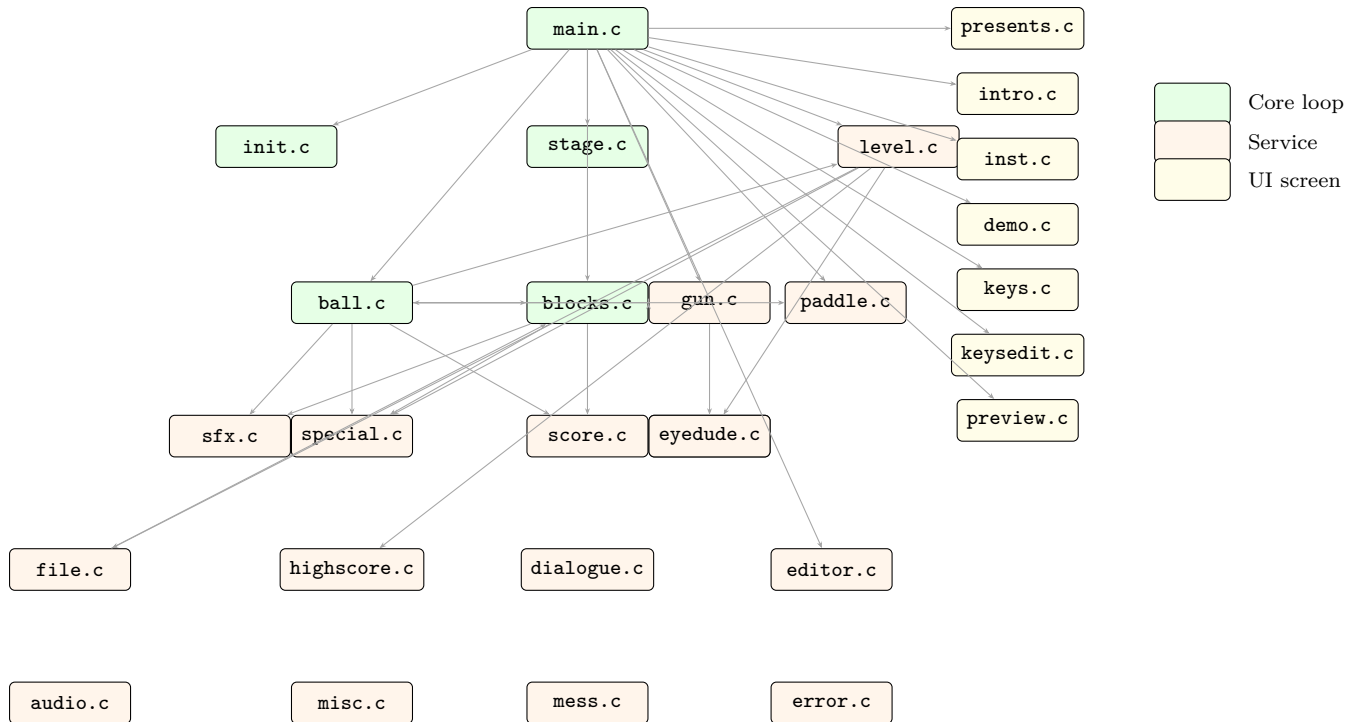


Figure 6: Module dependency graph. Arrows indicate “calls into” relationships. `main.c` dispatches to all modules via the state machine. `ball.c` is the most coupled module (6 outgoing dependencies). UI screen modules (yellow) share a common pattern: each has a multi-state sequence driven by `handleGameStates()` in `main.c`.

Table 18: Codebase metrics (original source, excluding tests and stubs).

Metric	Value
Source files (.c)	29 (27 + 2 generated)
Header files (.h)	33
Audio driver files	12
Source LOC (excl. tests)	18,606
Header LOC	2,953
Audio driver LOC	2,882
Total LOC	24,441
<code>extern</code> declarations	62
Game modes	16
Block types	30
Level files shipped	83
Sound files	46
XPM bitmap directories	11

Initialization order dependency

The 33-step initialization sequence in `init.c` has implicit ordering constraints. Reordering steps (e.g., creating GCs before windows) produces segfaults with no compile-time protection.

15 Known Issues and Technical Debt

The following issues motivated the modernization effort. Each is a factual observation about the legacy codebase, not a design criticism—many were reasonable choices for 1993.

Table 19: Technical debt inventory with modernization rationale.

Issue	Details
X11 server-side collision	Block collision regions (<code>XPolygonRegion</code> , <code>XRectInRegion</code>) are computed on the X server. Game logic should not depend on a display server. Replacing with ~ 20 lines of client-side triangle math removes this dependency.
Binary save format	<code>saveGameStruct</code> is serialized via raw <code>fwrite()</code> with no byte-order conversion and no struct padding protection. Save files are not portable across architectures.
Binary score format	<code>highScoreEntry</code> uses <code>htonl()/ntohl()</code> for numeric fields but relies on struct layout for <code>name[40]</code> . Padding between fields can vary across compilers.
OSS audio via fork+pipe	The <code>/dev/dsp</code> device (OSS) is deprecated on modern Linux. The fork+pipe model prevents mixing multiple sounds. Volume control is unimplemented.
K&R prototype guards	The original headers wrapped function prototypes in <code>#if NeedFunctionPrototypes</code> for K&R C compatibility (~ 200 lines). These guards have been removed from the archived source.
Compile-time paths	File system paths are baked in via <code>-D</code> flags. The binary can only run from the repo root unless environment variables are set. Incompatible with FHS installation (<code>/usr/share/</code> , <code>/usr/games/</code>).
Hand-tuned frame timing	<code>sleepSync()</code> multiplies the delay by 300 (μs factor). The archived source labels this “tuned for modern hardware” (the original used a larger constant). Frame timing remains unpredictable across hardware. No vsync support.
Private colormap	The <code>PseudoColor</code> colormap causes color flashing when the window manager switches focus on modern <code>TrueColor</code> displays—the private colormap overwrites colors used by other applications.
XLFD font dependency	Adobe Helvetica bitmap fonts are disappearing from Linux distributions. The <code>fixed</code> fallback is visually degraded.

Issue	Details
<code>snprintf()</code> /fixed buffers	All <code>sprintf()</code> calls have been converted to <code>snprintf()</code> , but fixed-size buffers remain throughout. The three <code>-Wno-*</code> compiler flag suppressions silence warnings about format truncation and unused results from <code>snprintf()</code> .
No test infrastructure	Zero unit tests, no test framework, no deterministic replay. The only way to verify behavior is manual play-testing.
62 global variables	Cross-module state is shared via <code>extern</code> globals. This makes reasoning about state ownership difficult and prevents unit testing individual modules in isolation.

A Command-Line Options

Table 20: Complete command-line option table (Spec §1).

Option	Argument	Default	Effect
<code>-display</code>	<code><name></code>	<code>\$DISPLAY</code>	X display connection
<code>-help</code>	—	—	Print help, exit
<code>-usage</code>	—	—	Print usage, exit
<code>-version</code>	—	—	Print version, exit
<code>-setup</code>	—	—	Print path info, exit
<code>-sync</code>	—	off	X protocol synchronization
<code>-debug</code>	—	off	Debug mode
<code>-sound</code>	—	off	Enable audio
<code>-nosfx</code>	—	SFX on	Disable visual effects
<code>-keys</code>	—	mouse	Keyboard paddle control
<code>-grab</code>	—	off	Grab pointer
<code>-noicon</code>	—	icon on	Disable custom icon
<code>-usedefcmap</code>	—	private	Use default colormap
<code>-scores</code>	—	—	Print scores, exit
<code>-speed</code>	<code><1-9></code>	5	Game speed
<code>-startlevel</code>	<code><1-80></code>	1	Starting level
<code>-maxvol</code>	<code><1-100></code>	0	Max volume
<code>-nickname</code>	<code><name></code>	real name	Player name (max 20 chars)

B Initialization Sequence

The 33-step initialization order from `InitialiseGame()` in `init.c` (Spec §16):

1. Parse command-line arguments
2. Open X display
3. Configure display (sync, error handlers, signals, random seed)
4. Select color visual (PseudoColor → DirectColor → TrueColor)
5. Create/configure colormap
6. `SetUpAudioSystem()` (if audio enabled)
7. Map named colors (8 colors): `InitialiseColourNames()`
8. Create windows: `CreateAllWindows()`
9. Create 6 graphics contexts: `InitialiseGraphics()`

10. Load 4 fonts (with “fixed” fallback): `InitialiseFonts()`
11. Set window backgrounds from XPM pixmaps: `SetBackgrounds()`
12. `InitialiseMessageSystem()`
13. `InitialiseBlocks()` — block grid and pixmaps
14. `InitialiseBall()` — ball animation frames
15. `InitialiseBullet()` — gun/bullet pixmaps
16. `InitialiseScoreDigits()`
17. `InitialiseLevelInfo()` — level info display
18. `InitialisePaddle()` — paddle pixmaps
19. `InitialiseDialoguePixmaps()`
20. `InitialiseEyeDudes()`
21. `SetUpPresents()`
22. `SetUpKeys()`
23. `SetUpKeysEdit()`
24. `SetUpInstructions()`
25. `SetUpIntroduction()`
26. `SetUpBonus()`
27. `SetUpHighScore()`
28. Create color cycling arrays (`reds[7]`, `greens[7]`)
29. Display initial level info and time
30. Draw specials panel
31. Select input events on main and play windows
32. Map main window
33. Install colormap

Shutdown (`ShutDown()`) reverses this sequence: free all module resources, free GCs, release fonts, free colormap, close display.

C Resource Lifecycle Pairs

Table 21: Setup/Free function pairs for all modules (Spec §16).

Module	Setup	Free
Audio	<code>SetUpAudioSystem()</code>	<code>FreeAudioSystem()</code>
Stage	<code>CreateAllWindows()</code>	<code>FreeBackgroundPixmaps()</code>
Blocks	<code>InitialiseBlocks()</code>	<code>FreeBlockPixmaps()</code>
Ball	<code>InitialiseBall()</code>	<code>FreeBall()</code>
Gun	<code>InitialiseBullet()</code>	<code>FreeBullet()</code>
Score	<code>InitialiseScoreDigits()</code>	<code>FreeScoreDigits()</code>
Level	<code>InitialiseLevelInfo()</code>	<code>FreeLevelInfo()</code>
Paddle	<code>InitialisePaddle()</code>	<code>FreePaddle()</code>
Dialogue	<code>InitialiseDialoguePixmaps()</code>	<code>FreeDialoguePixmaps()</code>
EyeDude	<code>InitialiseEyeDudes()</code>	<code>FreeEyeDudes()</code>
Presents	<code>SetUpPresents()</code>	<code>FreeAllPresents()</code>
Keys	<code>SetUpKeys()</code>	<code>FreeKeyControl()</code>
KeysEdit	<code>SetUpKeysEdit()</code>	<code>FreeKeyEditControl()</code>
Instructions	<code>SetUpInstructions()</code>	<code>FreeInstructions()</code>
Introduction	<code>SetUpIntroduction()</code>	<code>FreeIntroduction()</code>
Bonus	<code>SetUpBonus()</code>	<code>FreeBonus()</code>
HighScore	<code>SetUpHighScore()</code>	<code>FreeHighScore()</code>
Messaging	<code>InitialiseMessageSystem()</code>	<code>FreeMessageSystem()</code>

Module	Setup	Free
Editor	SetUpEditor()	FreeEditor()
Misc	(loaded in init.c)	FreeMisc()
Graphics	InitialiseGraphics()	ReleaseGraphics()
Fonts	InitialiseFonts()	ReleaseFonts()